

A proposal for solving all of the contracts concerns

Abstract

We have had a couple of attempts to resolve the Romanian NB comment about the lack of guaranteed-enforced contracts. We've also had some suggestions (e.g. in D3894R0) for a different design for Contracts, but there were some major concerns about it (reliance on lambdas for deferring evaluation for 'ignore', concerns about generic preambles/postambles).

This proposal proposes a (significant, but not a revamp) tweak to our contracts facility and makes the situation such that

1. the facility is simpler and easier to understand in its 'default' form and also in its non-default forms
2. the facility is more library-g geared, and by extension, extensible
3. guaranteed-enforced contracts can be easily expressed with it
4. the P2900 semantics can be easily expressed with it (well, almost all of them, see later for constification)
5. those audiences who have reported that they can't use the C++26 facility yet and need labels can build what they need in C++26, and we can later standardize better building blocks for those needs.

The lofty goal of the proposal is to address all the grievances reported, without giving up the capabilities that P2900 provides. All in C++26.

Syntax and semantics

Let us begin. First, consider the P2900 forms of

```
void f(int x) pre(x >= 0);  
int f() post(r: r >= 0);  
contract_assert(x);
```

In this approach, those are to be spelled

```
void f(int x) pre(std::pre(x >= 0));  
int f() post(r: std::post(r >= 0));  
contract_assert(std::cassert(x));
```

If you spell them in the current status quo (P2900) form, what you get is the 'enforce' semantic (without constification, without predicate exception translation). Except with this twist: if those expressions have `std::ignore` as their result type, you get 'ignore' instead.

This particular aspect of this proposal allows us to

1. avoid lambdas as a mechanism for deferring execution
2. avoid relying on a generic preamble/postamble mechanism

In other words, the language `_assertion_` facility has only two `_evaluation_` semantics:

1. enforce, and
2. ignore.

The language still has all of the P2900 semantics available as `_configurable_` semantics. But we don't actually need to specify a configuration mechanism in C++26, we can do that later. We then provide standard library functions that use an implementation-defined configuration mechanism, and we can make it use the standardized one in C++29.

In other words, the way it works is thus:

1. if a predicate expression's result type is the type of `std::ignore`, the assertion is evaluated with the 'ignore' semantic
2. otherwise, if the predicate's result is false (a bool), the violation handler is invoked, and if it returns normally, the program is contract-terminated.

In addition to that, we add the library parts:

3. `std::pre()` is a standard library function that queries an implementation-defined configuration facility to determine which semantic the precondition assertion should have, and then,
 - A. returns `std::ignore` if the semantic is 'ignore'
 - B. if the semantic is 'observe', and the expression evaluates to false or throws an exception, calls the violation handler and then returns true to the language facility (if we want, we can make it return a tag object that wraps the bool and signals to the core language part that 'enforce' is off). And then does nothing additional.
 - C. if the semantic is 'quick_enforce', and the expression evaluates to false, contract-terminates the program.
 - D. if the semantic is 'enforce', and the expression returns false or throws an exception, calls the violation handler and then contract-terminates the program..
4. `std::post()` is a standard library function that queries an implementation-defined configuration facility to determine which semantic the postcondition assertion should have, and then,
 - A. does the same things as the `pre()` does.
5. `std::cassert()` is a standard library function that queries an implementation-defined configuration facility to determine which semantic the assertion statement should have, and then,
 - A. does the same things as the `pre()` and `post()` do.

There is no need to change the provision to allow N evaluations of a non-ignored predicate, or to guarantee exactly-once-evaluation. This proposal concedes that point. QoI will do what's expected there.

What we end up with is that

1. Unless an explicit opt-in in the form of returning a `std::ignore` from a predicate is used, contract assertions are always-evaluated and always-enforced.
2. The P2900 semantics can be used by calling standard library functions that provide those semantics. Except for constification. More on that in a bit.
3. Completely custom semantics can be used by calling a function that does whatever it needs to do, predicate exception translation or not, violation handler call or not, call of some other handler, whatever.
4. Audiences who have reported that they need the configurability of something like P3400 can do what they need, today, without having to wait for the standard library facility. Works like this:
 - A. use a custom violation handler, have that call a function that you provide for your domain/organization/team
 - B. use custom assertion functions that can, when they need to, just call the aforementioned domain/org/team-specific function directly, or have them just let the core language facility call the violation handler, and that then calls the domain/org/team function.

5. **Portable** programs can't, in C++, access the compiler's configuration for contract evaluation semantics. But **programs** in general can; they can use the implementation-defined configuration mechanism directly. And then, in C++29, we can provide a standardized mechanism for accessing that configuration.
6. Extending the facility can be as simple as just using custom functions. More complex schemes like labels are not **required**. However, this approach doesn't preclude adding a label-like facility, or even P3400 almost as is. What is, however, suggested here, is that it's tweaked to follow the approach here, and that instead of

```
void f(int x) pre<some_label>(cond);
```

it will use something like

```
void f(int x) pre(std::pre<some_label>(cond));
```

instead. Which is something that can be added as an overload of `std::pre()`.

Constification

Let's just not do constification as a hard yes/no choice that we can't change, and instead supply a language mechanism (perhaps in C++29) for it:

```
constify(expression)
```

You can then use it anywhere. Instead of, or in addition to

```
if (cond) {}
```

you can write

```
if (constify(cond)) {}
```

and you can write

```
std::pre(constify(cond))
```

and those who can use macros, and will likely use macros anyway, can write

```
#define BSLS_PRE(cond) std::pre(constify(cond))
```

and have constification be the default for their organization, where they mandate the use of `BSLS_PRE` instead of using `std::pre` directly. (The `BSLS`-prefix is completely fictional. The naming style used in it is the work of the author's imagination. Any resemblance to actual naming schemes is entirely coincidental.)

Diagnostic quality for violations

Sharp-eyed readers will notice that, when not ignored, an invocation of a library function will most likely lose the knowledge of what the original expression was, as the library function is passed just a `bool`.

I expect this to be solved by the standard library implementation having a defaulted `std::source_location` parameter and then using an implementation-defined mechanism for filling the comment data of the contract violation based on that location information.

Providing a similar facility for portable custom assertions is left for future standards. Non-portable ones can of course again choose to use the implementation-define mechanism directly.

And yes, I know that the suggested implementation-defined facility will not work across modules or across TUs. But it seems plausible that we can solve that problem, perhaps by introducing a new facility that simply captures

code in a nearby context in text form, so that that text of the context can then be used by all sorts of facilities that need to print expressions like `static_assert` and current contract assertions do, and then we have succeeded in exposing that ability not to just individual core language facilities decided on a case-by-case basis, but also for user code. Like we're supposed to, if we follow our own design principles.

Why do this now, and what we end up gaining, as a summary

The reason this is being worked on are OMDB (and, OMNB) objections to what's in the draft now. It may have consensus, but it also has vehement objections, it causes severe heartburn. Despite the consensus, we as a committee are at war with some of our vendors and some of our significant members by railroading the current status quo forwards.

We don't have to be. There are plausible design alternatives that are palatable to those opposing audiences. Like this one seems to be.

It continues to be a much preferable approach to postpone contracts a bit, perhaps with the aid of a White Paper, and entertain these design alternatives. We're not practically changing anything by pushing the current contracts straight into the IS and refusing to look at alternatives. The implementations will be experimental anyway, and we can't then change what we standardized. No matter what feedback we get and how much. But failing that White Paper request, at least we could make a serious effort at standardizing something that causes less or even none of that heartburn.

Yes, we have deadlines, and we have processes, but the situation is extraordinary. The amount and severity of that heartburn is extraordinary. We have never had this much and this strong opposition to a feature in a DIS.

So what do we gain by adopting this approach? Several things:

- the semantics aren't ephemeral, or hard-coded into language magic. A contract assertion evaluates an expression, and that expression can be a function call. Use different functions to get different semantics; that's the simplest and most familiar extension mechanism ever. No need to define classes that model various complicated concepts, just call a different function. If someone wants to, those classes that model complicated concepts can also be used, but that's not something everyone has to use.
- thus adding new semantics and new functionality is simple and straightforward.
- the evaluation semantics used are discoverable in plain code.
- we have both always-enforced contracts and flexible contracts, and they both fit simply into a simple model.
- the default semantic, expressed with the shortest syntax, is simple and easy to understand. It doesn't have multiple semantics, unless explicitly asked for. It doesn't have four different semantics.